

# Computer Science & IT



## Basics of DBMS

Lecture No. 05



By- Vishal Sir



# Recap of Previous Lecture

- ✓ **Topic** Decomposition of relation
- ✓ **Topic** SQL commands and clauses
- Topic** Aggregate functions
- ✓ **Topic** Order of execution





# Topics to be Covered



✓  
Topic

Nested queries

✓  
Topic

IN, ANY, ALL and EXISTS operators

✓  
Topic

Transaction & Schedule

✓  
Topic

ACID properties

✓  
Topic

Classification of schedule

✓  
Topic

"B tree" and "B + tree"





## Topic : SQL clauses

- ✓ **FROM** - The FROM clause in SQL is used to select the database tables
- ✓ **WHERE** - The WHERE clause in SQL is used to retrieve the data from the database based on conditions specified with WHERE clause.  
*table*
- ✓ **GROUP BY** - GROUP BY clause is used to group the result of WHERE clause. *{If where clause is not present then Group by clause will be applied on complete table}*
- ✓ **HAVING** - HAVING clause can be used in a GROUP BY clause. It is used to specify a search condition for a group in the database tables.
- ✓ **ORDER BY** - The ORDER BY clause in SQL is used for sorting the records of the database





## Topic : SQL clauses



**FROM:-** From clause is used to select the tables from the database.

### STUDENT

| Sid | Sname | Marks | Branch |
|-----|-------|-------|--------|
| S1  | A     | 40    | CS     |
| S2  | A     | 20    | IT     |
| S3  | B     | 60    | CS     |
| S4  | A     | 60    | EC     |
| S5  | C     | 40    | IT     |
| S6  | C     | NULL  | EC     |

Select \* From STUDENT

Complete STUDENT table  
will be produced as o/p





## Topic : SQL clauses

**WHERE:-** Used to retrieve the data from the database based on conditions specified with WHERE clause.

*Everything*  
Select \* from STUDENT  
Where (Branch = 'CS')

Select Sid, Sname from STUDENT  
Where (Branch = 'EC')

### STUDENT

| Sid | Sname | Marks | Branch |
|-----|-------|-------|--------|
| S1  | A     | 40    | CS     |
| S2  | A     | 20    | IT     |
| S3  | B     | 60    | CS     |
| S4  | A     | 60    | EC     |
| S5  | C     | 40    | IT     |
| S6  | C     | NULL  | EC     |





## Topic : SQL clauses

**GROUP BY:-** GROUP BY clause is used to group the result of WHERE clause.

Query: Retrieve names of all branches along with maximum marks in that branch.

Select Branch, Max(Marks)  
from STUDENT  
Group By (Branch)

| Branch | Max(Marks) |
|--------|------------|
| CS     | 60         |
| IT     | 40         |
| EC     | 60         |

## STUDENT

| Sid | Sname | Marks | Branch |
|-----|-------|-------|--------|
| S1  | A     | 40    | CS     |
| S2  | A     | 20    | IT     |
| S3  | B     | 60    | CS     |
| S4  | A     | 60    | EC     |
| S5  | C     | 40    | IT     |
| S6  | C     | NULL  | EC     |



## Topic : Order of execution



Order of Execution:-

- ✓ 1. From
- ✓ 2. Where
- ✓ 3. Group By
- ✓ 4. Having
5. Select
6. Order BY





## Topic : SQL clauses



### NOTE:-

1. We can not select any attribute in SELECT clause along with aggregate function until those attributes are present in GROUP BY clause.
2. If aggregate function is used along with GROUP BY clause, then aggregate function is applied on each group.





## Topic : SQL clauses



**HAVING:-** HAVING condition is applied on each group.

Query: Retrieve branch names with average marks more than or equal to 40.

Select Branch From STUDENT

Groupby (Branch)

Having (Avg(Marks)  $\geq$  40)

$\Rightarrow$  o/p = 

|        |
|--------|
| Branch |
| CS     |
| EC     |

Having Clause is used to select the group which satisfy the condition specified with Having Cond<sup>n</sup>

| Sid | Sname | Marks | Branch |
|-----|-------|-------|--------|
| S1  | A     | 40    | CS     |
| S2  | A     | 20    | IT     |
| S3  | B     | 60    | CS     |
| S4  | A     | 60    | EC     |
| S5  | C     | 40    | IT     |
| S6  | C     | NULL  | EC     |





## Topic : SQL clauses



### NOTE:-

1. WHERE condition is applied on each tuple whereas HAVING condition is applied on each group.
2. We can use HAVING condition without GROUP BY clause, but in that case HAVING condition will be applied on each tuple.  
i.e., without GROUP BY clause HAVING clause will degenerate into WHERE clause.





## Topic : SQL clauses



**ORDER BY:-** This clause is used to sort the result in ascending or descending order based on values of attribute specified with ORDER BY clause.

By default

We need to specify

By default order is ascending order.

### STUDENT

| Sid | Sname | Marks | Branch |
|-----|-------|-------|--------|
| S1  | A     | 40    | CS     |
| S2  | A     | 20    | IT     |
| S3  | B     | 60    | CS     |
| S4  | A     | 60    | EC     |
| S5  | C     | 40    | IT     |
| S6  | C     | NULL  | EC     |





## Topic : Nested queries

(Subquery)



→ query within query

Nested Query

Independent  
Nested Query

Correlated nested query





## Topic : Independent nested query

eg: Select Sids of the students with maximum marks

Select Sid  
From STUDENT  
Where (marks = 60 Select Max (Marks) From STUDENT)

outer  
query

inner query

\* Inner query will be executed first,

→ It can be executed independently  
∴ Independent Nested query





## Topic : Correlated nested query



$R(A, B, C, D), S(L, M, N, O)$

eg

Select R.A  
From R

Where

operator

{ Select \*  
From S

Where (S.L = R.B) }

Correlated Nested query

Attribute from outer Rel<sup>n</sup>.

A subquery is called Correlated if for the execution of inner query the attribute from the relation specified in outer query is required.





## Topic : Correlated nested query

Order of execution:-

eg

⑥ — Select R.A

① — From R

⑤ — Where operator

{ Select \*  
From S  
Where (S.L = R.B) }

② ③ & ④  
Will be repeated on complete inner relation for each tuple of outer relation





## Topic : Operators

Operators which can be used with nested query are,

- ① IN
- ② ANY
- ③ ALL
- ④ EXISTS





## Topic : IN operator

"IN" operator returns true iff tuple under Consideration is present in set of tuples   
 *it is actually the o/p of inner query*

eg.  $x \text{ IN } \{1, 7, 3, 9, 5\}$

if  $x=4$ , then IN operator will return false

if  $x=3$ , then IN operator will return True



Note: If set of tuples is empty, {i.e. If o/p of inner query is empty} then "IN" operator always return false.

Note: Complement of "IN" is "NOT IN"





## Topic : ANY operator

"ANY" operator works along with other Comparison operator.

ie:  $<, >, \leq, \geq, =, <>$

\* "ANY" operator returns true iff . at least one tuple from the set of tuples satisfy the Comparison Condition w.r.t. given tuple.

eg.  $X \leq \text{ANY } \{ \overset{\times}{1}, \overset{\checkmark}{7}, 3, 9, 6 \}$

if  $X=10$ , then "ANY" will return false

if  $X=5$ , then "ANY" will return True



Note: "IN" is equivalent to "= ANY"

Note:- If inner query output is Empty then "ANY" operator will always return false.





## Topic : ALL operator

→ "ALL" operator will also work along with other Comparison operator

"ALL" operator returns false iff <sup>ie,  $<, >, \leq, \geq, =, <>$</sup>  at least one tuple from the given set of tuples fails the Comparison Cond<sup>n</sup> with concerned tuple.

eg.  $x > \text{ALL } \{1, 7, 3, 9, 6\}$

if  $x=10$ , then ALL returns true

if  $x=7$ , then ALL will return false



Note: "ALL" operator will always return true if inner query result is Empty.

NOTE: "NOT IN" is Equivalent to "<>ALL"





## Topic : EXISTS operator

- "EXISTS" operator returns true if and only if inner query result is not Empty.
- Complement of "EXISTS" is "NOT EXISTS"





## Topic : Transaction



It is a sequence of logically related operations.

eg:  $\neq$  Transfer Rs. 500/- from account A to account B

Transaction

Read (A) : Read the current value of A from DB into main mem

$A = A - 500;$  } this op<sup>n</sup> is performed in M.M. only,  $\therefore$  A in database is still = 1000.

Write (A) Write new value of A into database, after this Write (A) op<sup>n</sup>, A = 500 in DB as well

Read (B)

$B = B + 500$

Write (B)

Commit : Commit represents that transaction Executed Successfully

Not imp. w.r.t. DB.

let initially  $\left. \begin{array}{l} A = 1000 \\ B = 0 \end{array} \right\}$

There can be multiple transactions in the system

eg.

T<sub>1</sub>

R<sub>1</sub>(A)

W<sub>1</sub>(A)

R<sub>1</sub>(B)

W<sub>1</sub>(B)

}

T<sub>2</sub>

R<sub>2</sub>(A)

R<sub>2</sub>(B)





# Topic : Schedule



Time ordered sequence of operations of two or more transaction is called schedule

eg  $S_1$  :  $R_1(A)$ ,  $R_2(A)$ ,  $W_1(A)$ ,  $R_1(B)$ ,  $W_1(B)$ ,  $R_2(B)$

$S_2$  :  $R_1(A)$ ,  $W_1(A)$ ,  $R_2(A)$ ,  $R_1(B)$ ,  $W_1(B)$ ,  $R_2(B)$

time increasing ↓

| $S_1$ |          |          |
|-------|----------|----------|
|       | $T_1$    | $T_2$    |
| 1000  | $R_1(A)$ |          |
|       |          | $R_2(A)$ |
| 500   | $W_1(A)$ |          |
| 0     | $R_1(B)$ |          |
| 500   | $W_1(B)$ |          |
|       |          | $R_2(B)$ |

incorrect

| $S_2$ |          |          |
|-------|----------|----------|
|       | $T_1$    | $T_2$    |
| 1000  | $R_1(A)$ |          |
| 500   | $W_1(A)$ |          |
|       |          | $R_2(A)$ |
| 0     | $R_1(B)$ |          |
| 500   | $W_1(B)$ |          |
|       |          | $R_2(B)$ |



There are two types of schedules

① Serial schedule : Start the execution of a new transaction only after the complete execution of previously started transaction

Output of serial schedule is always correct, But with serial schedules throughput of the system may be low.

② Concurrent schedule: Operations of two or more transactions can execute in interleaved manner

Throughput increases with Concurrent schedule, but output of Concurrent schedule may be incorrect.





## Topic : ACID properties

- ① Atomicity
- ② Consistency
- ③ Isolation
- ④ Durability

{ ACID properties must be followed for integrity of database & transaction }





Atomicity: Either execute all the operation of a transaction or none of them

{ Problem is that a transaction may fail  
in between

∴ Recovery management Component must be used to ensure atomicity }

Equivalent Schedule: - Two schedules are said to be equivalent if their behaviour is same

Criteria for Equivalence: For two schedules to be equivalent

- ① Every read must be same in both the schedules.
- ② Final update of Every dataitem (if any) should also be similar in both the schedules.



Serializable Schedule: A schedule is said to be a serializable schedule iff it is Equivalent to at least One of the serial schedule { over the same transactions }

Non-Serializable Schedule: If given schedule is not Equivalent to any of the serial schedule, then it is called a non-serializable schedule.



$T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

Serial Schedule

$T_1 \rightarrow T_2$

$T_1 \rightarrow T_2$

| $T_1$           | $T_2$          |
|-----------------|----------------|
| 1000 - $R_1(A)$ |                |
| 500 - $W_1(A)$  |                |
| 0 - $R_1(B)$    |                |
| 500 - $W_1(B)$  |                |
|                 | $R_2(A) - 500$ |
|                 | $R_2(B) - 500$ |

Serial Schedule

$T_2 \rightarrow T_1$

$T_1$  |  $T_2$

|                 |                 |
|-----------------|-----------------|
|                 | $R_2(A) - 1000$ |
|                 | $R_2(B) - 0$    |
| 1000 - $R_1(A)$ |                 |
| 500 - $W_1(A)$  |                 |
| 0 - $R_1(B)$    |                 |
| 500 - $W_1(B)$  |                 |

| $S_1$           |                 |
|-----------------|-----------------|
| $T_1$           | $T_2$           |
| 1000 - $R_1(A)$ |                 |
|                 | $R_2(A) - 1000$ |
| 500 - $W_1(A)$  |                 |
| 0 - $R_1(B)$    |                 |
| 500 - $W_1(B)$  |                 |
|                 | $R_2(B) - 500$  |

Neither Equivalent to  $T_1 \rightarrow T_2$   
Nor Equivalent to  $T_2 \rightarrow T_1$   
 $\therefore$  Non-serializable

| $S_2$           |                |
|-----------------|----------------|
| $T_1$           | $T_2$          |
| $R_1(A) - 1000$ |                |
| $W_1(A) - 500$  |                |
|                 | $R_2(A) - 500$ |
| $R_1(B) - 0$    |                |
| $W_1(B) - 500$  |                |
|                 | $R_2(B) - 500$ |

$S_2$  is equivalent to serial schedule  $T_1 \rightarrow T_2$   
 $\therefore$  Serializable Schedule



② Isolation: - If two or more transactions are executing concurrently, then they all must be unaware of each other.

i.e. they must never come to know that they are executing along with other transactions.

To ensure isolation condition, a non-serializable schedule should never be allowed to execute.

Durability: - All the updation performed by a successful transaction must persist. even if system fails at later point of time.



Consistency :- Before and after the execution of transactions the database should remain consistent

Criteria to ensure Consistency:-

- ① Schedule must be Strict recoverable
- ② Schedule must be serializable





## Topic : Classification of schedule

Based on recoverability

- ① Irrecoverable Schedule
- ② Recoverable Schedule { Cascading Rollback & lost-update is possible }
- ③ Cascadeless Rollback Recoverable Schedule { No Cascading Rollback, but lost update possible }
- ④ Strict Recoverable Schedule { No Cascading Rollback & No lost-update }

Based on serializability

- ① Conflict Serializable Schedule
- ② View Serializable Schedule





→ To Ensure Serializability, Concurrency Control Component is used.

→ Concurrency Control Component can implement

lock based protocols {eg. Two phase Locking protocol}  
(2PL)

(or)

Time stamp base protocols {eg. Basic Time Stamp ordering protocol  
to Ensure Serializability & Time Stamp ordering protocol with  
Thomas Write Rule





## Topic : Index file



→ Index files are used to reduce IO Cost

→ There are two categories of indexes.

(i) Dense index { In dense index, one index entry per record of the database file. }

{ i.e. No. of entries in dense index file = No. of records in the database file }

(ii) Sparse index:-

{ For a set of database records there will be only one entry in index file }

in general, one index file entry per database block

{ i.e. No. of entries in sparse index file = No. of disk blocks required to store database file }



# Types of index:

There are three types of indexes.

Search Key  
DB file

| Id |  |
|----|--|
| 1  |  |
| 2  |  |
| 3  |  |
| 4  |  |
| 5  |  |
| 6  |  |
| 7  |  |

① Primary index { when search key attribute values are unique in database file, and records of the file are in sorted order based on search key attribute values }  
{ may be dense or sparse }  
(generally sparse)

Search Key  
DB file

| Dept. No | --- |
|----------|-----|
| 1        |     |
| 2        |     |
| 3        |     |
| 4        |     |
| 5        |     |
| 6        |     |
| 7        |     |

② Clustering Index (Always sparse) { when search key attribute values are not unique in database file, but still records of the file are in sorted order based on search key attribute values }

③ Secondary index { Search key attribute values may or may not be unique }  
(Always dense) { but records of the file are not ordered based on search key value }

Not Unique  
But records are ordered based on Dept. No.



If database file is too large, and single level index file does not provide significant improvement w.r.t. reduction in IO cost, then we may use multi-level index.

eg. Balanced Search Trees.

- ① B Tree
  - ② B+ Tree
- } This are multi-way search tree }





## Topic : B tree



The order of a node of B tree is defined as maximum number of child pointer it can have.

→ If 'm' is the order of a node of B tree, then

- ① Maximum no. of child pointer a node can have = m
- ② Maximum no. of keys a node can have = (m-1)
- ③ Minimum no. of child pointer a non-root node can have =  $\lceil \frac{m}{2} \rceil$
- ④ Minimum no. of keys a non-root node can have =  $\lceil \frac{m}{2} \rceil - 1$
- ⑤ Minimum no. of child pointer a root node can have = 2
- ⑥ Minimum no. of keys a root node can have = 1





## Topic : B + tree



In B+ tree order of a leaf node and order of a non-leaf node is defined differently.

Order of non-leaf node:- order of a non-leaf node is defined as maximum number of child pointer a non-leaf node can have

Order of leaf node:-

Order of a leaf node of B+ tree is defined as Maximum number of keys a leaf node can contain



Note:

- ① In B-tree all nodes <sup>{internal + leaf}</sup> will store key values and record pointer corresponding to those key values.
- ② In B+ tree internal node will only hold the key values and those key values are dummy keys which are used just for search operation.

In B+ tree all the keys along with their record pointers are stored at leaf nodes.





## 2 mins Summary



✓ Topic

Nested queries

✓ Topic

IN, ANY, ALL and EXISTS operators

✓ Topic

Transaction & Schedule

✓ Topic

ACID properties

✓ Topic

Classification of schedule

✓ Topic

"B tree" and "B + tree"



**THANK - YOU**